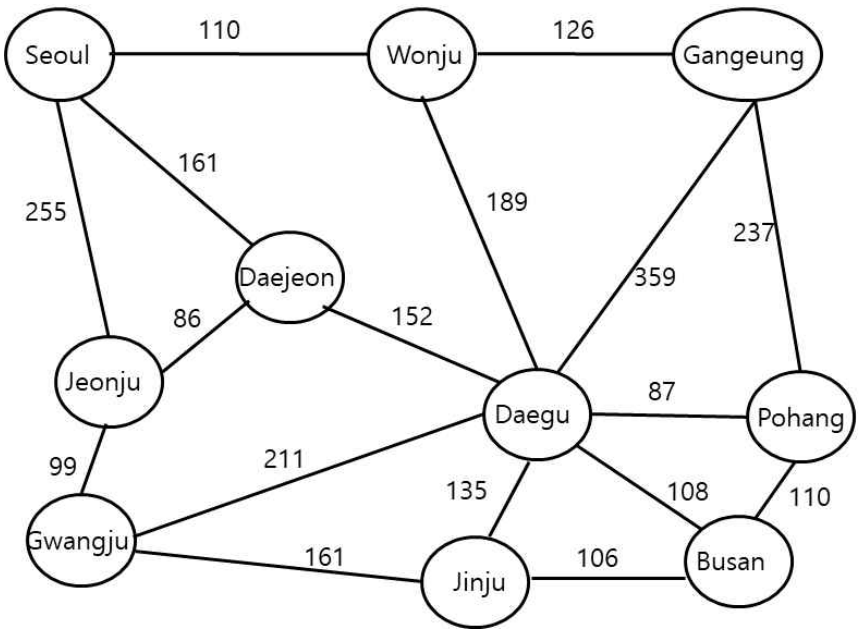
Algorithm Analysis Homework 6

Due by 6/13(Fri.)

Write a C++ program to solve the all-pairs shortest path problem using the following algorithms:

* Dijkstra's Algorithm: Apply Dijkstra's algorithm |V| times, once for each vertex.
* Floyd-Warshall Algorithm: Implement Floyd's algorithm to compute the shortest paths.

Sample graph is as follows.



[Input file format]:

* The input graph is stored in a file named homework6.data.
* The file represents the graph using an adjacency matrix format.
* Assume that there is exactly one tab (₩t) between matrix values.
* The number of vertices in the graph is at most 20.
* If there is no direct route between two cities, the distance is represented as "INF".

[Program Outline]

* Read the input file (hw6.data).
* Construct the graph:
  - Store it in an adjacency matrix and adjacency list.
* Run Dijkstra's Algorithm:
  - Apply Dijkstra's algorithm |V| times (once per vertex).
  - Print the shortest distances between all pairs of cities.
* Run Floyd-Warshall Algorithm:
  - Compute and print the shortest distances using Floyd's algorithm.
* Handle Special Cases:
  - Test the program with graphs containing negative-weight edges and negative-weight cycles.
  - Note that Dijkstra's algorithm does not work correctly with negative weights, but Floyd-Warshall can handle them (except for negative cycles).

[Sample output]

1) The shortest distance between cities using Dijkstra's algorithm is:

	Busan	Daegu	Daejeon	Gang neung	Gwang ju	Jeonju	Jinju	Pohang	Seoul	Wonju
Busan	0	108	..	..	..	..	..	110	..	297
Daegu	108	0	..	..	..	..	..	..	..	..
Daejeon	..	..	0							
Gang neung	..	..	..	0	..	..	..	..	..	..
Gwang ju	..	..	..	..	0	..	..	..	..	..
Jeonju	..	..	..	..	..	0	..	..	..	..
Jinju	..	..	..	..	..	..	0	..	..	..
Pohang	110	..	..	..	..	..	..	0	..	..
Seoul	..	..	..	..	..	..	..	400	0	..
Wonju	297	..	..	..	..	..	..	..	..	..

(Replace .. with actual computed distances.)

2) The shortest distance between cities using Floyd's algorithm is:

/* a table with the same format as the table above. */

[Testing]

* Test the program with graphs containing:
  - No negative-weight edges (to verify correctness).
  - Negative-weight edges (to observe differences in behavior between the two algorithms).
  - Negative-weight cycles (to confirm detection, as Floyd's algorithm cannot compute valid paths in such cases).
* Note: This testing is optional and will not be graded, but it helps you understand the algorithm differences.

[Submission Guidelines]

* Late submissions will not be accepted. If you submit after the due date, you will receive zero points.
* Ensure your output is neatly formatted for clarity.
* The program must be written in C++, and you may use any C++ STL features.
* The program must compile and run using g++. If it does not compile, you will receive zero points.
* Test your program thoroughly using the provided example graph and additional test cases.
* Do not use large-scale AI tools to generate your solution.
* Include a header comment in your code listing all references used, such as

  For ex)

  (1) 강의 slide chapter 16.

  (2) Blog: ** URL here **

  (3) Book: "Algorithm analysis in C++" by Someone